

Listing_Describe_Solution

AppExchange Listing — “Describe Your Solution”

Field on the Partner Console submission form: *Describe Your Solution* — Provide a detailed technical description of your solution and related components. Optionally, include links to documentation that you want to share with our reviewers. You can also attach documents on the Upload Documentation page.

Submitted for: Portwood DocGen Managed v2.0.0 (04tVx000000ZqBpIAK), 2026-05-22 re-submission

Paste-ready text (plain, no markdown rendering)

Portwood DocGen is a 100% native Salesforce document generation engine with built-in electronic signatures. The package generates PDFs, Word documents, Excel spreadsheets, and PowerPoint presentations by merging live Salesforce data into user-uploaded Office Open XML templates. All processing occurs entirely within the Salesforce platform — no external callouts, no Remote Site Settings, no Named Credentials, no third-party JavaScript libraries, no CDN fetches, no session-ID usage, and no hardcoded secrets.

Architecture summary. The package ships 41 non-test Apex classes (28 test classes, 1,436 tests at 76% org-wide coverage), 18 Lightning Web Components, 4 Visualforce pages, 11 custom objects (9 sObjects + 2 platform events), and 4 permission sets. The Apex layer is split between thin @AuraEnabled / @InvocableMethod controllers (DocGenController, DocGenBulkController, DocGenChartImageController, DocGenSetupController, DocGenSignatureSenderController, DocGenSignatureController, DocGenAuthenticatorController) and a service layer (DocGenService merge engine, DocGenDataRetriever V1/V2/V3 SOQL, DocGenHtmlRenderer for OOXML→HTML, the v1.99 PNG chart pipeline classes, and DocGenSignatureService for async signed-PDF finalization). Asynchronous work runs through Batchables (DocGenBatch, DocGenGiantQueryBatch), Queueables (chart imaging, giant-query stitching, signature PDF generation), and a Schedulable (signature reminders). Platform-event triggers decouple guest-context signing from the system-context PDF/email pipeline.

Security model. Admin endpoints run with sharing, gated by the DocGen_Admin / DocGen_User / DocGen_Guest_Runner permission sets. Every admin @AuraEnabled / @InvocableMethod entry point opens with an explicit Schema.sObjectType.isAccessible | isCreateable | isUpdateable() gate — the documented enforcement signal the AppExchange sfge:ApexFlsViolation rule pattern-matches on, and the reviewer's explicit "first alternative" from the v1.56 finding-resolution language. Standard objects (ContentVersion, ContentDocumentLink, User, OrgWideEmailAddress) use WITH USER_MODE throughout. Package-namespaced custom-object operations use WITH SYSTEM_MODE behind the Schema gate, because USER_MODE strict-FLS strips namespaced custom fields when subscriber admin profiles haven't been granted FLS individually per release (reproducible in the package-build context). Guest signing endpoints run without sharing and route through a new DocGenSignatureGuestSecurity helper class that centralizes Schema-CRUD describe checks, SHA-256 hex token-shape validation, and per-operation field allowlists. Signing is gated by a 64-character SHA-256 capability token (single-use, 48-hour expiry) plus an email-PIN second factor (SHA-256-hashed at rest, 10-minute expiry, 3-attempt lockout).

This re-submission (v2.0.0, package version 04tVx000000ZqBpIAK). The AppExchange security review of the v1.56 listing returned 30 findings (4 clickjacking, 26 CRUD/FLS). v2.0.0 closes every one of them, extends the same hardening proactively to code the reviewer didn't flag, and ships one verifier bug fix (the document verifier now returns the complete multi-signer audit trail instead of LIMIT-1 truncating to the first signer). v2.0 also rolls forward ~44 versions of feature work since v1.56 — V3 visual query trees, the pure-Apex PNG chart engine (9 styles, no external rendering), signature v3 with multi-signer + sequential/parallel order + per-tag guided placements + in-person signing, HTML templates with embedded image extraction, giant-query batching for multi-million-row child datasets, and document-hash verification.

Salesforce Code Analyzer (Security + AppExchange rule selectors): 0 Critical / 0 High / 38 Moderate (all documented false positives — see DocGen_Code_Analyzer_Report and DocGen_False_Positive_Report uploaded with this submission).

Attached documentation (see Upload Documentation page):

- DocGen_Solution_Architecture_and_Usage — security-focused architecture, threat model, sharing model, controls matrix
- DocGen_Architecture_and_Usage — feature/component inventory and usage walkthroughs
- DocGen_Platform_Technology — Salesforce platform technology inventory (this answers the "If your solution contains Salesforce Platform technology" follow-up question)
- DocGen_Code_Analyzer_Report — sf code-analyzer run results with finding dispositions
- DocGen_False_Positive_Report — Checkmarx CxSAST false-positive disposition with v2.0 hybrid-pattern rationalizations
- SECURITY_REVIEW_RESPONSE_v2 — per-finding map of the 30 v1.56 findings to the specific v2.0 commits/files that resolve them

- GitHub repository: <https://github.com/Portwood-Global-Solutions/DocGen> (release tag v2.0.0)
-

Word count

~580 words. The Partner Console field accepts long technical descriptions, but this is short enough that a reviewer skimming will reach the v2.0 re-submission paragraph quickly.

Related forms

- For the **“If your solution contains Salesforce Platform technology”** follow-up, attach `DocGen_Platform_Technology.md` from this folder. That doc is the detailed Apex / LWC / VF / object / permset / external-tech inventory.
 - For the **“What is the security architecture?”** style follow-ups, attach `DocGen_Solution_Architecture_and_Usage.md`.
-

*File maintained at docs/appexchange/v2.0.0/
Listing_Describe_Solution.md so future re-submissions can pull this verbatim
or update in place.*